

Risposte quiz simulazione 29-05-2025

1. L'ambiente (o environment) è (e) L'insieme delle associazioni (nome, entità denotabile) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma
2. Le affermazioni vere sugli oggetti denotabili vere sono:
 - (b) Un oggetto denotabile è un oggetto a cui può essere associato un nome
 - (e) I nomi delle variabili sono esempi di nomi definiti dagli utenti
3. Le caratteristiche di un comando nei linguaggi imperativi sono
 - (a) modifica lo stato
 - (b) può avere side effects
4. Le affermazioni vere sulle binding policy sono:
 - (a) la binding policy è ortogonale (indipendente) alla scope policy (indipendentemente da cosa è lecito fare)
 - (c) nello shallow binding l'environment da considerare è quello attivo al momento della chiamata della funzione passata come parametro
 - (d) la binding policy entra in gioco quando abbiamo funzioni di ordine superiore
5. La mutua ricorsione tra funzioni (a) è la ricorsione che occorre tra due o più funzioni quando si richiamano reciprocamente.
6. Le affermazioni vere sulle funzioni di ordine superiore sono:
 - (c) La chiamata per nome è un caso particolare di quello delle funzioni passate per parametro
7. Dato il seguente pseudocodice

```
int y = 12;
int f1(int x)
{
    x = 12;
    y = 1;
    return x / 2;
}
int f2(void)
{
    int z, w;
    z = y / 3;
    return (f1(z)-z);
}
```

il valore di ritorno di f2 se i parametri sono passati per valore è (b) 2.
8. Una forma normale nel lambda calcolo è la forma in cui un'espressione non può essere ulteriormente beta-ridotta
9. Beta-riducendo $(\lambda x.x)(\lambda y.yy)(\lambda p.pq)$ si ottiene (c) qq.
10. In ML l'unica espressione legale è (c) (1,2,0);
11. Nel seguente programma ML:

```
fun pow x 0 = 1.0
  | pow x y = x * pow x (y-1);
```

il tipo della funzione pow 5.0 in ML è (c) fn: int -> real poiché abbiamo già passato alla funzione il reale (la base della potenza).
12. Dato il seguente codice Prolog

```
unknown1([A|_],A).
unknown1([_|T],A) :- unknown1(T,A).
unknown(X,Y,[X|T]) :- unknown1(T,Y).
unknown(X,Y,[_|T]) :- unknown(X,Y,T).
```

unknown (A,B,L) (c) controlla se A occorre in testa alla lista L (restituisce true se A occorre in testa a L)

13. Dato il frammento di programma (espresso in pseudo-codice) riportato di seguito, il valore di ritorno di h(), assumendo scoping statico, è (c) 1.

```
int x, y, z;
void f(void)
{
    int x;
    x = 1;
    y = 2;
}
void g(void)
{
    int z;
    int y;
    f();
    z = 6;
    x = 7;
}
int h(void)
{
    int x;
    x = 3;
    y = 5;
    g();
    z = x - y;
    return z;
}
```

14. Le affermazioni sul check dei tipi vere sono:

- (c) uno degli svantaggi del dynamic type checking è l'essere meno efficiente a runtime
- (e) i tipi sono utili non solo a livello concettuale ma anche in termini di controllo di correttezza del programma e a livello di implementazione